

Multithreading

CS358 Lecture Slides

Dr. Srishti Srivastava



Shifting a Core Assumption

- Up to this point, most - if not all - of your study of computer science has been based on the idea that ...
- Only one thing happens a time.
- This is known as **sequential programming** ...
- Where every operation is carried out as part of a single sequence

↓ Start end ↓

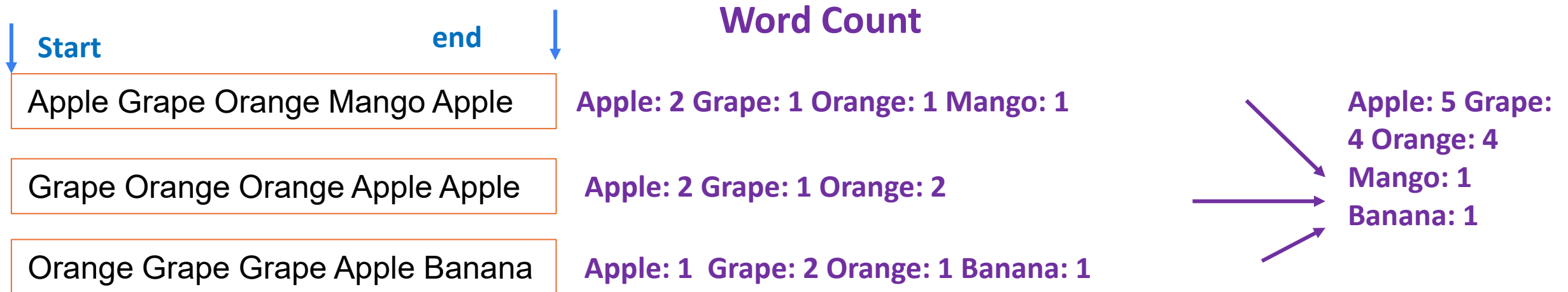
Apple Grape Orange Mango Apple Grape Orange Orange Apple Apple Orange Grape Grape Apple Banana

Word Count

Apple: 5 Grape: 4 Orange: 4 Mango: 1 Banana: 1

Shifting the Sequential Assumption in Programming

- Shifting from Sequential Programming to Parallel Programming
 - Break a task into multiple smaller tasks (threads) of execution
 - Coordinate and synchronize those threads



Why Multithreading?

- **Modern hardware is parallel**
 - CPUs have multiple cores
 - GPUs have thousands of cores
 - One core can execute multiple threads
- **Sequential programs underutilize hardware**
 - One thread = only one core working
 - Other stay idle -> wasted resources
- **Performance gains**
 - Divide big problems into smaller tasks
 - Run tasks simultaneously -> faster results

Parallelism vs. Concurrency

- **Parallelism:**
 - Multiple tasks executing simultaneously on multiple cores
- **Concurrency:**
 - Multiple tasks making progress at the same time
 - May be on a single core (switching quickly between threads)
- **In Java:**
 - Concurrency: Thread, ExecutorService
 - Parallelism: parallelStream(), Fork/Join Framework

Processor & Thread

Processor (CPU core)

- Hardware that executes instructions
- Multi-core CPU can run many tasks at once

Thread

- Smallest unit of execution in a program
- Multiple threads can run on different cores or share one core

Key Idea:

- Threads = software workers
- Processors = hardware that runs them

Overhead

Extra work required for multithreading



Sources:

Thread
creation/destruction

Context switching

Synchronization
mechanisms



Impact: Too many threads can slow a program instead of speeding it up

Speedup

Goal: Faster execution using more cores

Ideal: Speedup = # of processors (linear)

- Number of processors: p
- *Serial run-time* = T_{serial}
- *Parallel run-time* = T_{parallel}
- $T_{\text{parallel}} = T_{\text{serial}} / p$
- **Speedup** = $T_{\text{serial}} / T_{\text{parallel}}$

Reality (Amdahl's Law):

- Sequential portion of code limits maximum speedup
- Example: If 20% is sequential -> max speedup = 5x, no matter how many cores

Using multiple cores

- Essentially all computers now are multi-core (dual-core means 2 cores, quad-core means 4 cores, etc)
 - Each core can run all the operations needed to run a program
- Up to this point, our programs have always done a single thing at once so one can run on each core
- What if we wanted multiple cores to run a single program faster?
 - Need to split program into parts that can run on different cores

Sample problem: Summing an array

```
int simpleSum(int[] array, int low, int hi)
{
    int sum = 0;
    for(int i=low; i < hi; i++)
        sum = sum + array[i];
    return sum;
}
```

Let's try this the unplugged way...

Let's use the following array:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
17	12	04	21	07	66	90	11	16	18	10	22	29	56	78	98	65	43	75	51	33	89	53	50	44	81	49	84	32	14	06	03

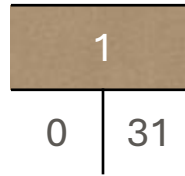
Unplugged activity

DIVISION OF WORK IN A POOL OF 15, WITH A CUTOFF OF 4 VALUES

- Sum numbers from 1 to 32 on a single post it note, using a pool of 15 workers, starting from one worker
- If range has more than cutoff values, split in half and give ranges to two workers using two separate post it notes
- Repeat recursively, with a parallel cutoff at a range of 4 numbers
- Below cutoff, workers serially sum numbers on their own post it notes, pass result (post it note) back to parent thread (join)
- When both threads join, parent node adds the two results and passes it up

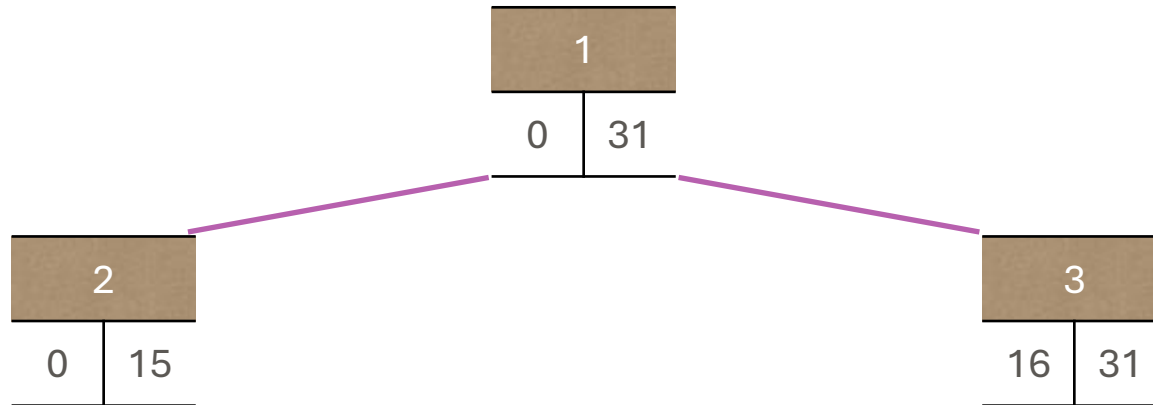
Division of work in a pool, with a cutoff

VISUALIZATION



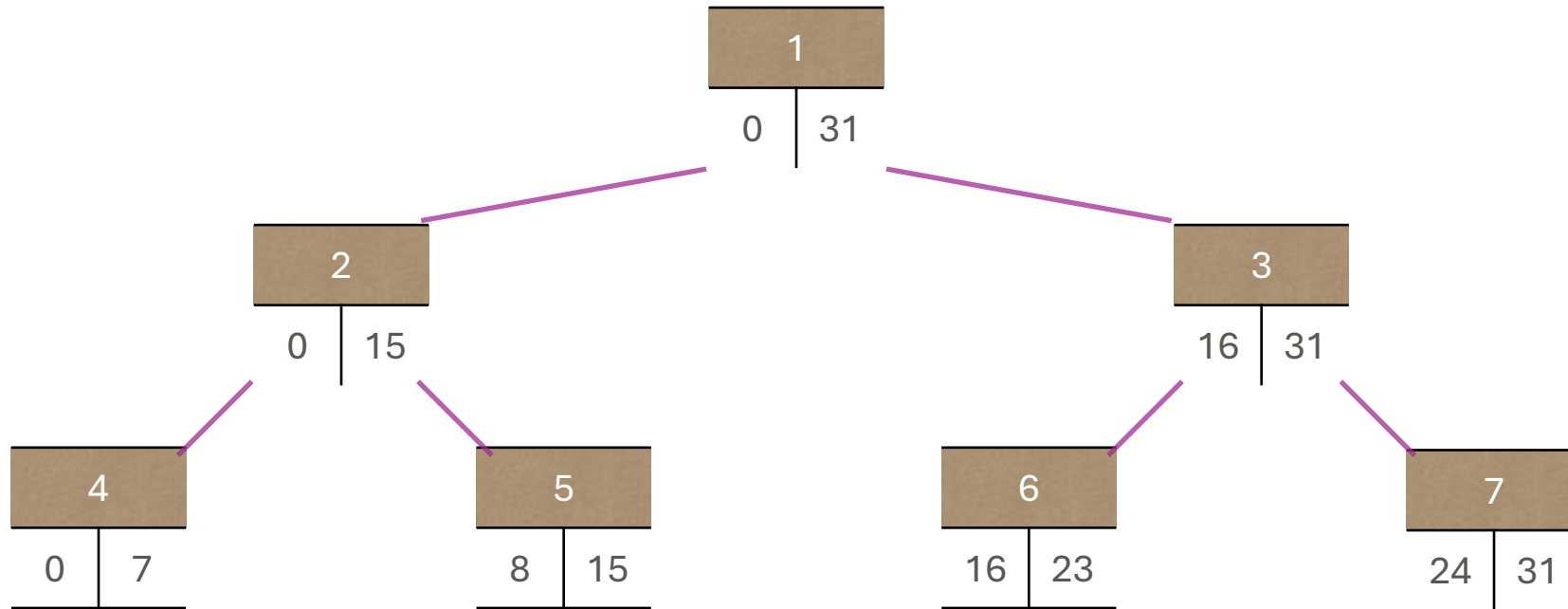
Division of work in a pool, with a cutoff

VISUALIZATION



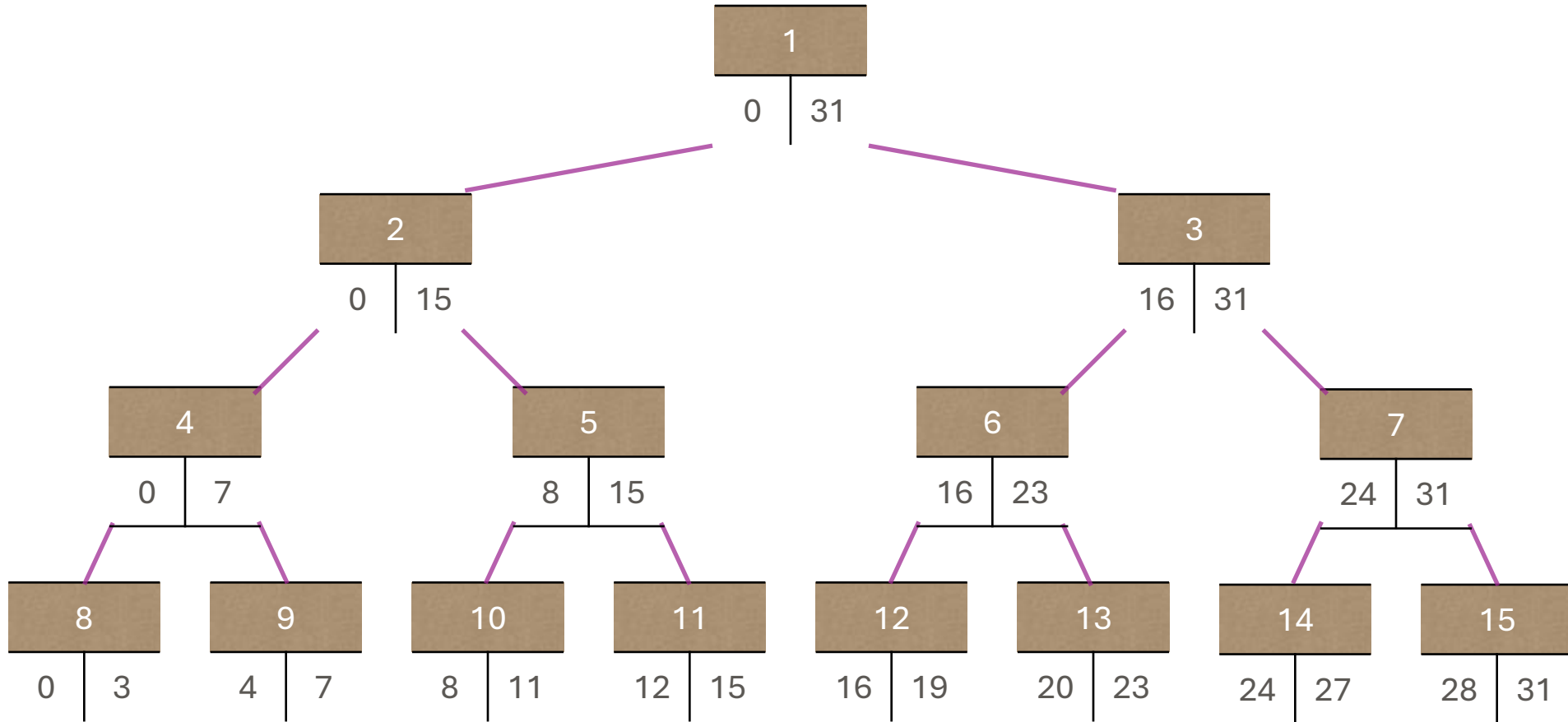
Division of work in a pool, with a cutoff

VISUALIZATION



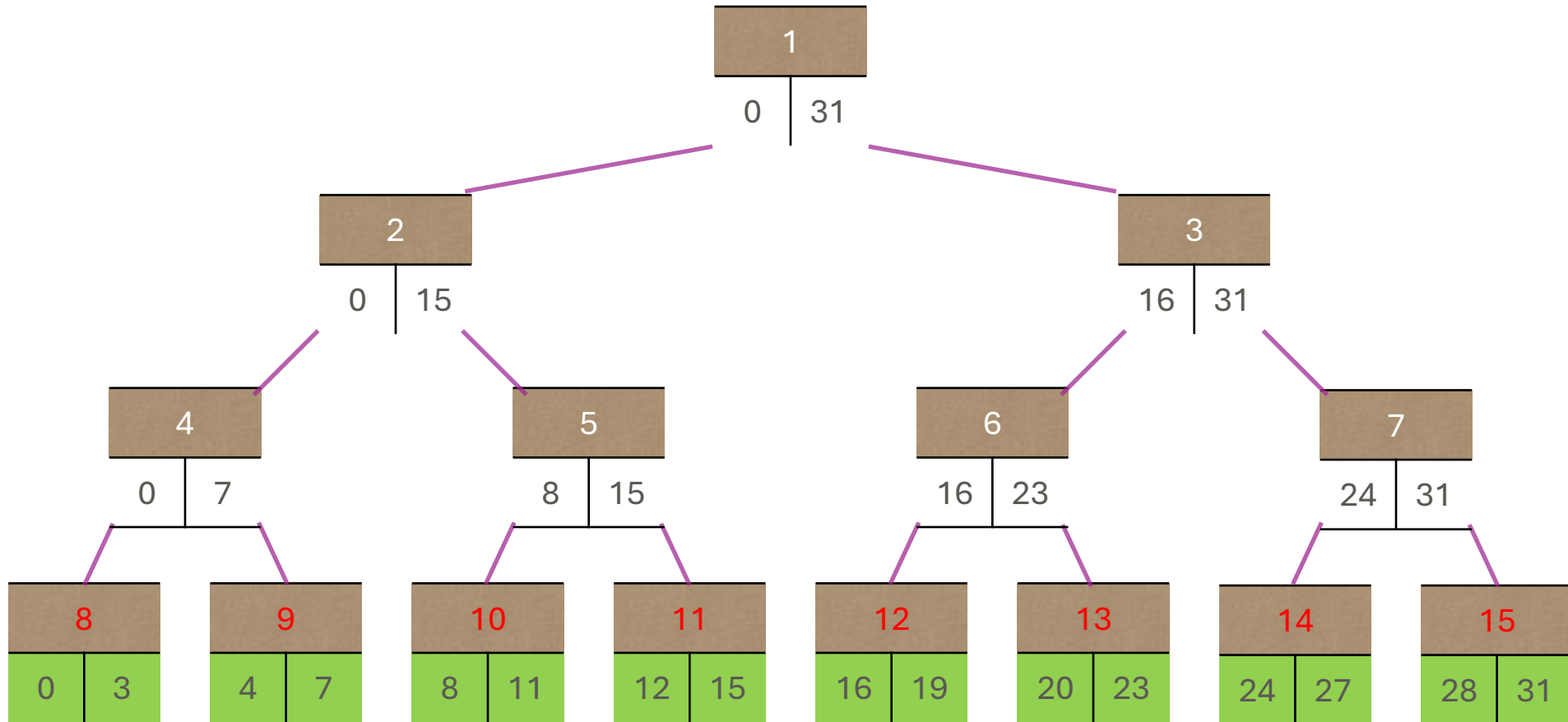
Division of work in a pool, with a cutoff

VISUALIZATION



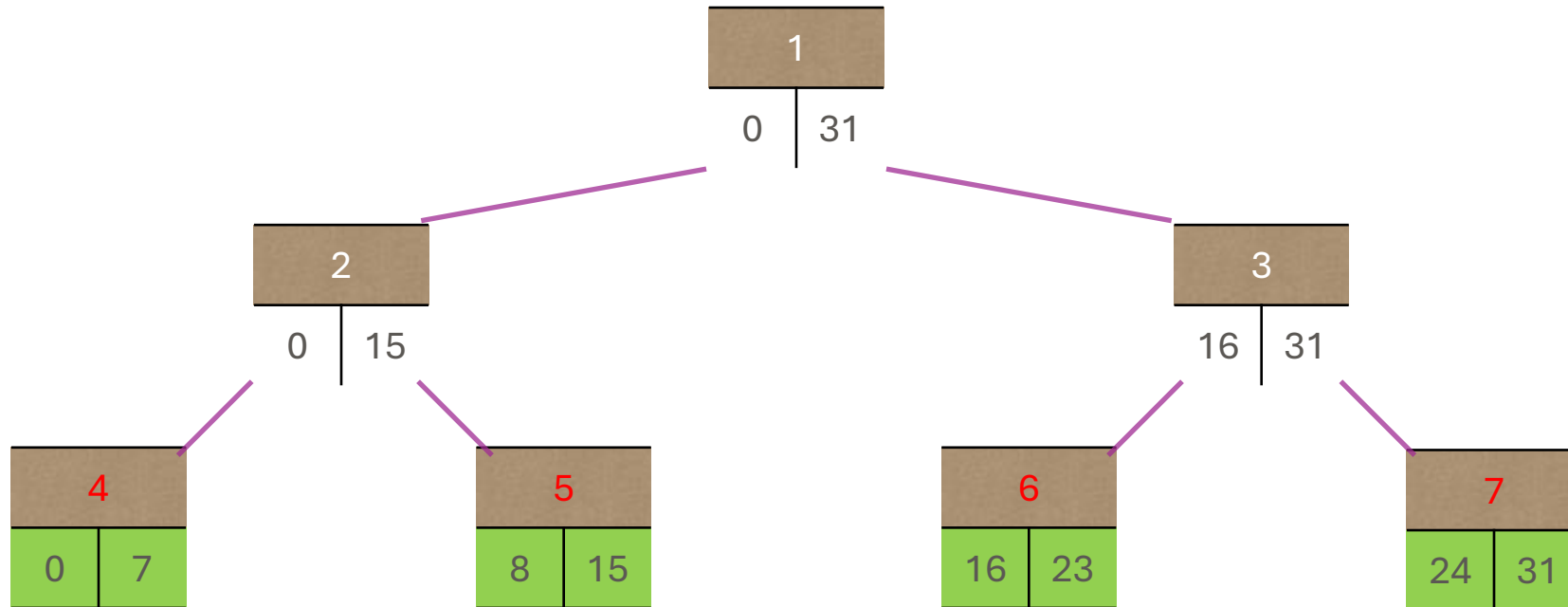
Division of work in a pool, with a cutoff

VISUALIZATION



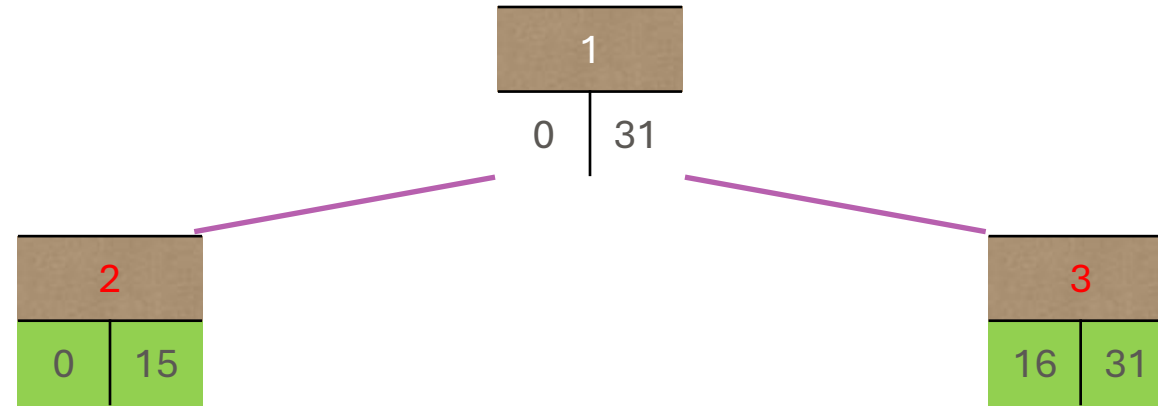
Division of work in a pool, with a cutoff

VISUALIZATION



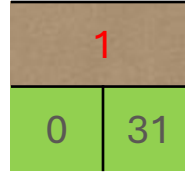
Division of work in a pool, with a cutoff

VISUALIZATION



Division of work in a pool, with a cutoff

VISUALIZATION



Division of work in a pool, with a cutoff

WHAT HAPPENED?

- There was a phase of dividing work
- Work-dividing threads then waited for joins
- Only the threads below the cutoff did real work (imagine a cutoff of 10,000)
- Work-dividing threads did a single computation and exited
- Seven of fifteen threads were mostly idle
- Can we make this better? Ideas?
- Let's Explore Java Thread class and the runnable interface

Let's revisit
the method

Splitting the
sum into two
parts

```
int sum(int[] array, int low, int hi)
{
    int left = simpleSum(array, low, (hi+low)/2);
    int right = simpleSum(array, (hi+low)/2, hi);
    return left + right;
}
```